

Supernode Detector: Automated Community Detection for Neuronpedia Attribution Graphs

Supernode Detector: Automated Community Detection and Annotation for Neuronpedia Attribution Graphs

Type: Tool paper **Status:** Working release **Repository:** github.com/J-Lawrence10/autocircuit (directory: `supernode_detector/`) **Last updated:** 2026-04-08

Abstract

We present **Supernode Detector**, an end-to-end tool for extracting, annotating, and publishing interpretable supernode communities from sparse autoencoder (SAE) attribution graphs. Given a text prompt, the tool generates a circuit graph via the Neuronpedia Circuit Tracer API, applies multi-algorithm community detection with a penalty-based selection score, trims each community by influence to a configurable percentage of the graph, labels communities using Neuronpedia feature explanations, and produces an annotated JSON ready for direct upload to the Neuronpedia interactive graph viewer. The tool supports batch processing of multiple prompts and programmatic upload via the Neuronpedia graph API, enabling researchers to go from prompt to published interactive visualization without manual intervention.

1. Motivation

Attribution graphs produced by tools like Neuronpedia's Circuit Tracer contain thousands of nodes and tens of thousands of edges per prompt, making manual interpretation infeasible. Existing community detection approaches (e.g., Louvain) require per-graph tuning, produce oversized or fragmented partitions depending on the graph's density, and do not integrate with downstream annotation or publication workflows. Researchers who want to share circuit analyses face a multi-step manual process: run clustering, label communities, format the output JSON by hand, upload via a web form, and repeat for each prompt.

The Supernode Detector addresses this gap with an opinionated, end-to-end pipeline that requires only a text prompt and an API key.

2. System Architecture

The pipeline consists of six sequential stages, all implemented in a single Python script (`scripts/supernode_pipeline.py`):

1. **Graph generation.** Calls `POST /api/graph/generate` on Neuronpedia and downloads the raw attribution JSON. An `--raw-graph` flag allows skipping this step when working with existing graphs.
2. **Community detection.** Runs three algorithms — Louvain, Leiden, and Greedy Modularity — on the graph's adjacency structure, then selects the best partition using a penalty-weighted modularity score (see §3.1).
3. **Recursive splitting.** Communities exceeding 30% of total graph size are recursively split using higher-resolution Louvain runs, capped at 10 sub-communities per oversized parent to prevent fragmentation.
4. **Bottleneck identification.** Computes betweenness centrality across the full graph to identify the top 10 critical routing nodes, which are pinned in the Neuronpedia viewer for high visibility.

5. **Semantic labeling.** For each community, queries the Neuronpedia feature API for the top 3 nodes by influence. Generates a descriptive label using either top-3 summarization (keyword overlap) or top-1 direct labeling, with a structural fallback when explanations are unavailable.

6. **Influence trimming and export.** Keeps only the highest-influence nodes per community, targeting a configurable fraction of total graph nodes (default 15%). Bottleneck nodes are always retained. Writes `annotated_graph.json` with `qParams.supernodes`, `qParams.pinnedIds`, and `metadata.feature_details`, plus a markdown report and a community visualization PNG.

3. Algorithmic Details

3.1 Penalty-Weighted Algorithm Selection

Naive modularity maximization can select algorithms that produce degenerate partitions — for example, Greedy Modularity sometimes returns 3 giant communities with a slightly higher modularity than Louvain's 11 balanced communities. To prevent this, the tool computes a penalized score for each algorithm:

```
score(alg) = modularity(alg) - 0.05 × n_oversized - 0.001 × n_singletons
```

where `n_oversized` counts communities exceeding 30% of graph size and `n_singletons` counts communities with fewer than 3 nodes. This keeps interpretable partitions competitive without discarding modularity as the primary metric.

3.2 Influence-Based Trimming

Raw community partitions include noise nodes that dilute interpretation. The tool applies a two-pass trimming procedure:

1. Compute a target node budget: `target_count = target_pct × n_nodes`.
2. Distribute the budget proportionally to each community's size, with a dynamic minimum of `max(1, min(3, int(avg_budget × 0.3)))` per community to avoid starving small but meaningful clusters.
3. Within each community, retain the top-influence nodes until the community's budget is filled.
4. Bottleneck nodes (from step 4 of the pipeline) are always retained regardless of community budget.

In practice, this reduces a ~1200-node graph to ~170-180 nodes (14-15% of the original) while preserving all critical routing structure.

3.3 Semantic Labeling Strategy

For each community with at least two annotated top features, the tool extracts keyword overlap across the explanations and builds a descriptive label. When only one explanation is available, it truncates the top feature's explanation to 35 characters and appends layer information. When no explanations are available (typical for QWEN features, which lack API support), a structural fallback produces labels like "L12-L18 cluster (42 nodes)".

4. Neuronpedia Compatibility and Upload

4.1 Cantor Pairing

Neuronpedia's graph viewer expects node feature IDs to be **cantor-paired** — a single integer encoding both the layer number and the SAE dictionary index. During export, the tool automatically transforms each node's raw feature value:

```
sae_index = raw_feature_id % 16384
cantor_feature = (layer + sae_index) * (layer + sae_index + 1) // 2 + sae_index
```

This ensures that clicking a node in the Neuronpedia viewer resolves to the correct feature dashboard. The transformation is applied transparently during the export step; the user does not need to manage it.

4.2 Programmatic Upload

The `--upload` flag implements Neuronpedia's three-step graph upload flow:

1. **Slug uniqueness.** The tool appends a Unix timestamp to the graph slug before upload, preventing 400 errors from slug collisions with other users' graphs.
2. `POST /api/graph/signed-put` with the filename, content length, and content type → returns a pre-signed S3 URL and `putRequestId`.
3. `PUT` the annotated JSON directly to the S3 URL (no authentication required for this step).
4. `POST /api/graph/save-to-db` with the `putRequestId` → returns the viewable graph URL.

Authentication uses the `x-api-key` header with the key loaded from `config/neuronpedia_config.yaml`. On success, the viewable URL is printed to stdout and saved to `upload_result.json` (including the slug and `putRequestId` for reference). On failure, the local file is preserved and instructions for manual upload are printed.

A `--upload-only` PATH flag enables uploading existing annotated graphs without re-running the pipeline.

Note: The Neuronpedia web validator's upload button may be disabled for certain models. The programmatic API works regardless — the tool bypasses the web UI entirely.

5. Batch Mode

The CLI accepts multiple positional prompts or a batch file:

```
python scripts/supernode_pipeline.py "The capital of France is" "The capital of Japan is" --upload
python scripts/supernode_pipeline.py --batch-file prompts.txt --upload
```

Batch files use one prompt per line with `#` for comments and blank lines skipped. Each prompt runs independently; a failure in one prompt does not abort the batch. After all prompts complete, the tool saves `batch_results.json` with per-prompt status, output directories, and upload URLs, and prints a summary table.

Combined with `--upload`, this enables a researcher to publish dozens of circuit analyses with a single command.

6. Validation

The tool was validated against the cross-domain circuit analysis pipeline (see companion paper, `CROSS_DOMAIN_CIRCUIT_PAPER.md`) on 60 circuits spanning chemistry, geography, and history prompts. End-to-end runs on sample prompts:

Prompt	Nodes (raw)	Communities	Nodes retained	Time
"The chemical symbol for water is"	1316	14	178 (13.5%)	~5 min
"The Titanic sank in the year"	1314	11	172 (13.1%)	~5 min
"The capital of France is"	1078	9	150 (13.9%)	~4 min

Semantic labeling achieves meaningful cluster names for GEMMA-2-2B. For example, the Titanic circuit produces labels including "words indicating periods or divisions" and "the phrase 'in order to'". QWEN circuits currently receive structural-only labels due to the Neuronpedia feature API not yet supporting QWEN SAEs.

7. Comparison to Existing Tools

Feature	Supernode Detector	Manual Louvain	Neuronpedia web UI
Multi-algorithm selection	Yes	No	No
Oversized community splitting	Yes	No	No
Influence-based trimming	Yes	No	No
Automatic semantic labeling	Yes	No	No
Bottleneck pinning	Yes	No	Manual
Batch processing	Yes	No	No
Programmatic upload	Yes	N/A	Manual
Dependencies	Python + Neuronpedia API key	NetworkX	Browser

The tool's value proposition is the integration of these capabilities into a single opinionated pipeline, not any individual capability.

8. Usage

```
# Install
pip install -r requirements.txt
# Add your API key to config/neuronpedia_config.yaml

# Single prompt
python scripts/supernode_pipeline.py "The capital of France is"

# Batch with upload
python scripts/supernode_pipeline.py --batch-file prompts.txt --upload

# Tweak trimming target
python scripts/supernode_pipeline.py "prompt" --target-pct 0.10

# Upload only (skip pipeline)
python scripts/supernode_pipeline.py --upload-only output/gemma-2-2b_my-prompt/annotated_graph.json
```

9. Limitations

- GEMMA-only semantic labeling.** Neuronpedia's feature explanation API does not yet support QWEN SAEs. QWEN circuits work end-to-end but receive structural labels instead of semantic ones.
- Single-model runs.** The tool processes one model per invocation; cross-model comparison requires running twice and merging manually.
- API rate limits.** Semantic labeling makes one API call per top-3 node per community; on large graphs this can take several minutes. A `--rate-delay` flag is provided for throttling.
- Community algorithm degeneracy.** Multi-algorithm validation in the companion paper shows that community assignments are algorithm-dependent (mean Jaccard agreement = 0.363 across Louvain, Leiden, Infomap, and Label Propagation). The tool's penalty scoring mitigates but does not eliminate this issue.
- No interactive preview.** Users cannot preview or rename communities before export; this is a planned future enhancement.

10. Availability

Source code: `supernode_detector/scripts/supernode_pipeline.py` (1,400 lines, MIT license)

Documentation: `supernode_detector/README.md`

Design

doc:

`supernode_detector/docs/supernode-pipeline-design.md`

Dependencies:

`networkx,`

`python-louvain, leidenalg, python-igraph, matplotlib, requests, pyyaml, numpy`

The tool is part of the broader autocircuit research project and shares infrastructure with the cross-domain circuit analysis pipeline described in the companion paper.

11. Acknowledgments

Built on the Neuronpedia platform (Decode Research) and uses the Circuit Tracer API infrastructure originally developed by Anthropic for attribution graph analysis.